

TÉLÉCOMMUNICATIONS ET TRAITEMENT DE SIGNAL

CODAGE CANAL

Saad BENJELLOUN HARZIMI
Aurélien VERBEKE

TABLE DES MATIERES

1	Introduction – Principe du codage canal	2
2	Codage canal par blocs linéaires	3
2.1	Codage	3
2.2	Décodage	3
2.3	Résultats obtenus	4
3	Codage canal convolutif	4
3.1	Codage	4
3.2	Décodage	5
3.3	Réponses aux questions	5
3.4	Résultats obtenus	6
4	Comparaison des deux méthodes	7

1 Introduction – Principe du codage canal

Le codage canal consiste à rajouter de l'information aux séquences à envoyer pour éviter les erreurs dans la chaîne de transmission, dues au bruit par exemple.

Nous étudierons ici le codage canal de deux types : par blocs et convolutif.

Le codage par blocs augmente les mots d'information avec des bits supplémentaires construits à partir d'opérations sur les bits initiaux.

Le codage convolutif applique une transformation convolutive sur les bits à envoyer.

Le décodage peut être fait de deux manières : sur les bits décodés après seuillage et *demapping*, c'est le décodage dur, ou directement sur le signal reçu : c'est le décodage souple.

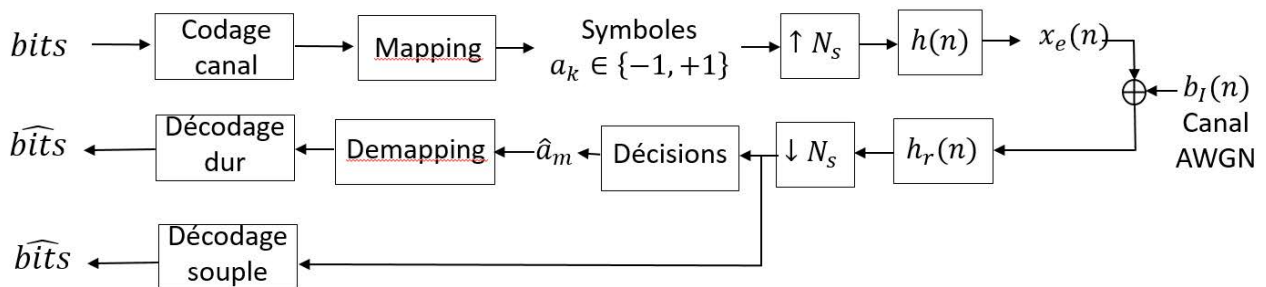


FIGURE 1 – Chaîne de transmission avec codage canal

2 Codage canal par blocs linéaires

2.1 Codage

Rappel : nous travaillons dans un espace modulo 2

On définit d'abord une matrice de codage G dont le rôle est de convertir les mots à envoyer initialement (i de longueur k) en mots « codés » (c de longueur n) qui seront transmis avant d'être décodés.

On peut en particulier choisir d'utiliser une matrice de code dit « systématique » : le mot initial apparaît dans le mot codé.

La forme d'une telle matrice est alors :

$$G = \begin{pmatrix} g_{1,k+1} & \cdots & g_{1,n} \\ I_k & \cdots & \cdots \\ g_{k,k+1} & \cdots & g_{k,n} \end{pmatrix} = (I_k \ P)$$

On a ainsi la relation entre un mot initial et un mot codé :

$$c = iG = [i \ iP]$$

Par la suite, nous utiliserons $k = 4$, $n = 7$ et la matrice de codage de Hamming suivante :

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

2.2 Décodage

Algorithme 1 : Décodage dur

Seuiller le signal reçu;

Démapper le signal en bits;

pour chaque mot de bits démappés faire

 Calculer sa distance de Hamming à tous les mots codés possibles;

 Choisir le mot de distance minimale comme mot décodé (il suffit de récupérer la partie d'indices 1 à k puisque le code est systématique);

La distance de Hamming est : nombre de 1 dans la somme $\oplus$ entre le mot à décoder et le mot comparé. Cela revient à compter le nombre de bits différents entre les deux mots.

Algorithme 2 : Décodage souple

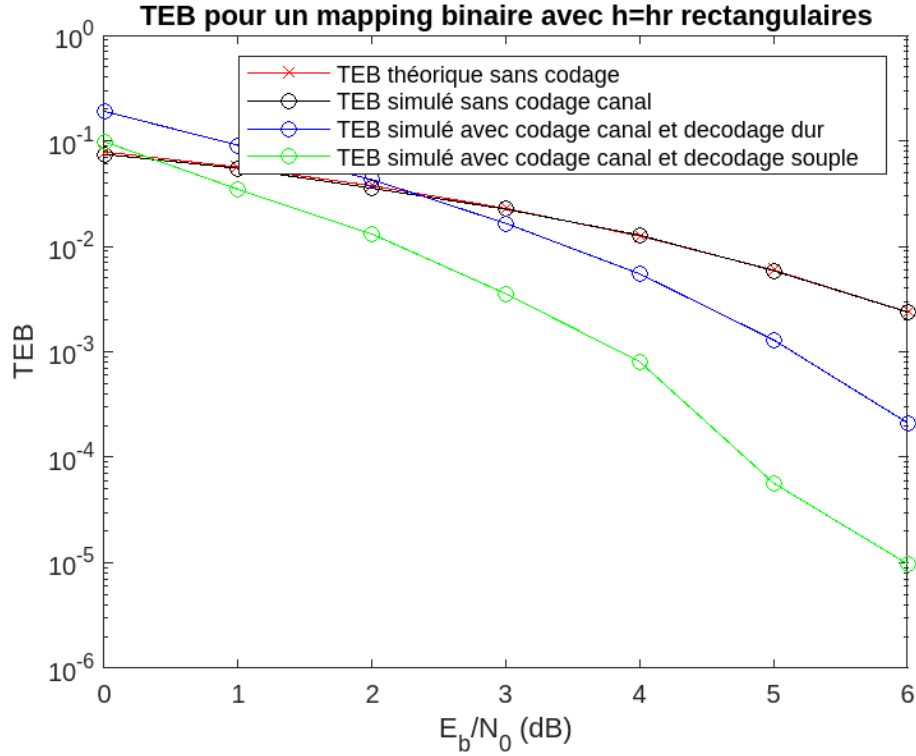
pour chaque mot de symboles bruts (non seuillés) faire

 Calculer sa distance euclidienne à tous les mots codés possibles;

 Choisir le mot de distance minimale comme mot décodé (il suffit de récupérer la partie d'indices 1 à k puisque le code est systématique);

Démapper les mots en bits;

2.3 Résultats obtenus



Il est à première vue clair que le codage canal avec décodage souple est bien plus efficace en termes d'efficacité en puissance. Il permet d'atteindre un TEB presque 1000 fois plus faible pour un rapport signal à bruit de 6dB et reste aussi très efficace à bas bruit.

Le décodage dur, lui, ne permet d'améliorer une chaîne sans codage qu'en présence d'un niveau de bruit minimum. En deçà de ce niveau minimum de 2,5dB, le TEB avec codage est moins satisfaisant que sans codage.

L'efficacité spectrale est, elle, moins bonne avec codage canal que sans.

On rappelle sa formule de calcul : $\eta = \frac{D}{B}$. La bande de passante est inchangée puisqu'elle dépend uniquement de la chaîne de transmission. Cependant, le débit binaire est diminué d'un rapport $\frac{n}{k}$ puisqu'on transmet à la fois les bits d'information et les bits de redondance.

3 Codage canal convolutif

3.1 Codage

Le codage canal convolutif consiste à transformer un bit en entrée par n bits en sortie, en prenant en compte les m bits en entrée précédents.

Ainsi chaque bit s_i sera calculé comme la somme $\oplus$ modulo 2 du bit en entrée (b_i) et de certains (ou tous) bits en entrée précédents ($b_{i-1}, \dots, b_{i-m}$).

On peut alors définir une matrice des bits à prendre en compte pour chacun des bits de sortie :

$$G = \begin{pmatrix} \text{bits pour la sortie 1} \\ \dots \\ \text{bits pour la sortie n} \end{pmatrix} = \begin{pmatrix} g(i-m, 1) & \dots & g(i, 1) \\ \dots & \dots & \dots \\ g(i-m, n) & \dots & g(i, n) \end{pmatrix}$$

Avec $g(a, b)$ l'opérateur binaire qui indique ou non la prise en compte du bit a dans la sortie b .

Ainsi, il suffit juste de convoluer la matrice avec la séquence de bits initiale pour obtenir la séquence de bits codée.

3.2 Décodage

Pour le décodage, on applique l'algorithme de Viterbi. L'avantage principal de celui-ci est qu'il offre une solution efficace sans devoir tester toutes les séquences possibles (ce qui serait très coûteux).

Algorithme 3 : Algorithme de Viterbi simplifié

pour chaque groupe de n bits ou symboles bruts reçus **faire**

 Calculer sa distance au groupe de bits ou symboles associés à chacune des transitions du treillis;

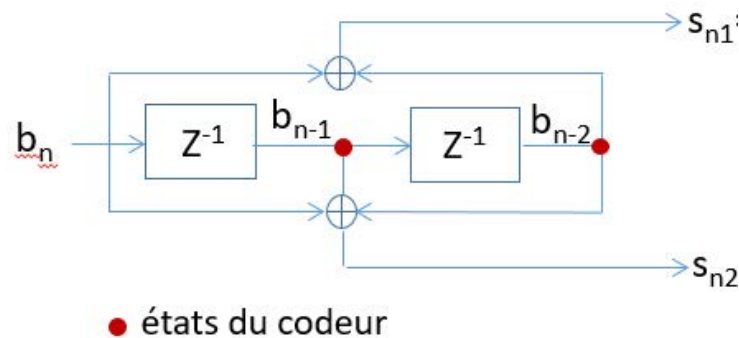
 Pour chaque nœud d'arrivée, conserver la transition qui permet d'obtenir la distance cumulée la plus faible.

Partir du nœud final de distance cumulée la plus faible et remonter le treillis total jusqu'au début.

En ce qui concerne la décision, on a toujours 2 types :

- **Décision dure** : On utilise la distance de Hamming sur les bits déduits du seuillage et démappage des symboles reçus.
- **Décision souple** : On utilise la distance euclidienne sur les valeurs du signal reçu brut.

3.3 Réponses aux questions



Question 1 : Quelle est la mémoire du code ?

La mémoire d'un tel code est $m = 2$.

Question 2 : Donnez les expressions de g_1 et g_2 définissant ce code dans les 3 représentations possibles (vecteur d'éléments binaires, octal, polynômes générateurs).

$$g_1 = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix} \text{ ou } g_1 = 5_8 \text{ ou } g_1 = 1 + X^2$$

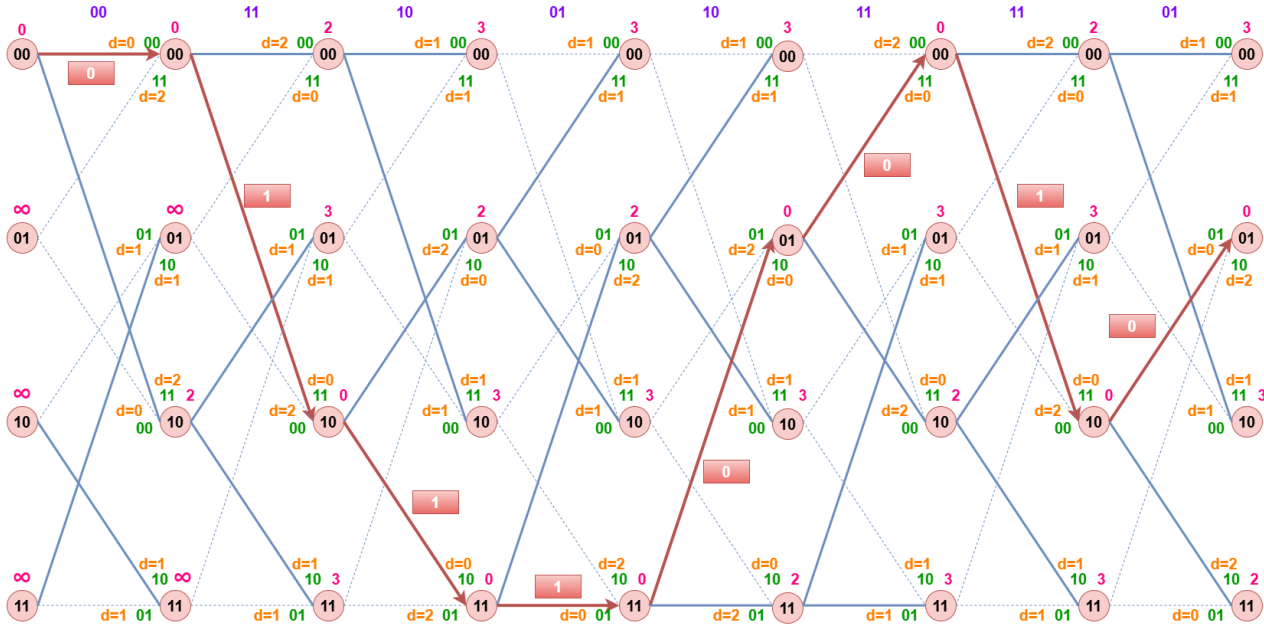
$$g_2 = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \text{ ou } g_2 = 7_8 \text{ ou } g_2 = 1 + X + X^2$$

Soit la séquence de bits émise suivante : [0 1 1 1 0 0 1 0]

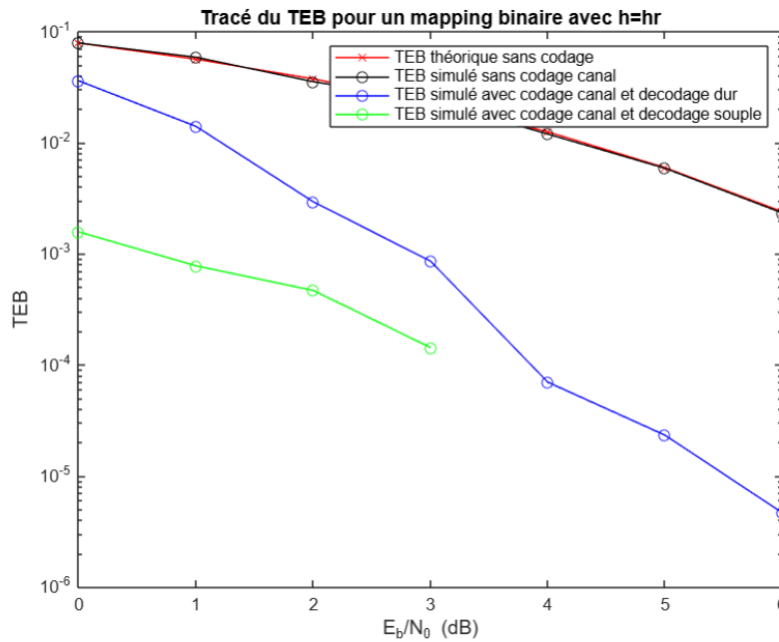
Question 3a : Déterminez la séquence de bits en sortie du codeur

La séquence codée est [0 0 1 1 1 0 0 1 1 0 1 1 1 0 1].

Question 3b : En supposant qu'il n'y a pas de canal de transmission, c'est-à-dire que la séquence de bits reçue à l'entrée du décodeur est égale à celle émise en sortie du codeur, donnez, comme dans l'exemple de la figure 5, le treillis associé à un décodage de Viterbi mais de type décisions dures.



3.4 Résultats obtenus



Le tracé du TEB en fonction du rapport $\frac{E_b}{N_0}$ montre très clairement l'impact bénéfique du codage convolutif combiné au décodage par l'algorithme de Viterbi.

Comparé à la chaîne sans codage, l'ajout du code convolutif permet de réduire considérablement le taux d'erreur. Le décodage Viterbi souple offre de meilleures performances que le décodage dur, en exploitant de manière plus fine les variations du signal reçu.

Ce codage permet donc un gain en efficacité en puissance significatif, au prix d'un coût en débit binaire et donc en efficacité spectrale (car le débit binaire est doublé à la sortie du codeur).

4 Comparaison des deux méthodes

En termes d'efficacité en puissance, le codage convolutif est meilleur : on atteint des TEB 10 fois meilleurs.

En termes d'efficacité spectrale, cela dépend des paramètres des codeurs par blocs et convolutif qui ont un impact direct sur le débit binaire (la bande utilisée, elle, ne change pas puisqu'elle dépend uniquement de la chaîne de transmission utilisée). Avec les paramètres utilisés ($k = 4$, $n = 7$ pour le codage par blocs, et $n = 2$ pour le codage convolutif), on a $\eta_{conv} = \frac{1}{2}\eta_{norm}$ et $\eta_{blocs} = \frac{4}{7}\eta_{norm}$ donc $\eta_{norm} > \eta_{blocs} > \eta_{conv}$.